DOCUMENTO TÉCNICO OFICIAL DE PRÁCTICAS

Laboratorio Avanzado de Ciberseguridad Defensiva, SIEM & Hardening de Redes

> Auditoría práctica integral, correlación de amenazas en tiempo real, detección de intrusiones, respuesta activa ante incidentes y blindaje perimetral de microservicios web.

8	8	+2,330	100%
LABS WAZUH SIEM	FASES DE RED & PROXY	EVENTOS ANALIZADOS	DEFENSA VERIFICADA

ESPECIALISTA RESPONSABLE
Mario Alvarez Ramos

INFRAESTRUCTURA DE EVALUACIÓN
Ubuntu Server 26.04 LTS (Docker) · Kali Linux · Tailscale Mesh VPN

HERRAMIENTAS CLAVE
Wazuh SIEM/XDR · Suricata NIDS · Nginx · iptables · Hydra · Nmap

FECHA & ESTADO
Septiembre 2026 · Pruebas de Seguridad Verificadas con Éxito

1. Resumen Ejecutivo & Topología del Laboratorio

> **Objetivo General:** Desplegar, auditar y defender una infraestructura moderna de servidores y microservicios mediante la implementación de un Centro de Operaciones de Seguridad (SOC) local basado en Wazuh SIEM/XDR, detección de red con Suricata y simulación de ataques reales desde Kali Linux.

El entorno de pruebas fue configurado bajo un esquema de **Zero Trust** y aislamiento de red. La intercomunicación entre el servidor de análisis, el servidor web y la máquina atacante se gestiona mediante una red privada cifrada de malla (Tailscale VPN), evitando la exposición indiscriminada de puertos de gestión hacia internet público.

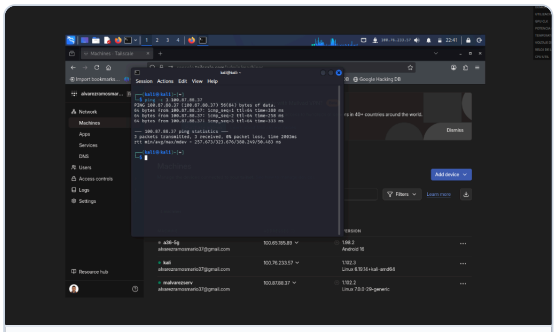
ARQUITECTURA DEL SISTEMA ● Red Cifrada Activa

Interconexión de Componentes

La infraestructura se divide en tres roles claramente delimitados:

- ✓ **Servidor SIEM / SOC (Ubuntu Server - 100.87.88.37):** Aloja el clúster de Wazuh (Manager, Indexer y Dashboard) sobre contenedores Docker. Procesa registros de eventos, correlaciona ataques y coordina respuestas activas.
- ✓ **Sensor de Red NIDS (Suricata Engine):** Inspecciona en tiempo real el tráfico de paquetes que entra y sale del host, identificando escaneos de puertos y patrones de explotación.
- ✓ **Máquina de Auditoría Ofensiva (Kali Linux - 100.76.233.57):** Ejecuta simulaciones de amenazas (fuerza bruta, escaneos agresivos, inyecciones web y pruebas de estrés DoS).

```
mario@malvarezserv:~$ sudo tailscale status
100.87.88.37 malvarezserv linux active; direct
192.168.1.50:41641
100.76.233.57 kali-security linux active; direct
```



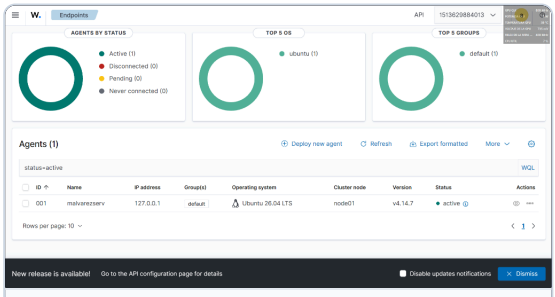
Evidencia 0.1: Topología de red cifrada mediante Tailscale Mesh VPN.

AGENTE DE ENDPOINT Sincronización Total

Monitorización Continua del Host

El agente de Wazuh instalado en el sistema operativo monitoriza continuamente los registros del sistema (`auth.log`, `syslog`, Docker containers logs), la integridad de archivos del sistema y la postura de configuración.

- ✓ Transmisión cifrada por el puerto seguro 1514 TCP hacia Wazuh Manager.
- ✓ Registro y decodificación en formato JSON para consultas ultrarrápidas en OpenSearch Indexer.
- ✓ Recolección centralizada de telemetría sin degradación del rendimiento de la CPU.



Evidencia 0.2: Agente Wazuh ID 001 ('malvarezserv') en estado 'Active'.

2. Módulo I: Ciberseguridad Defensiva con Wazuh SIEM & Suricata

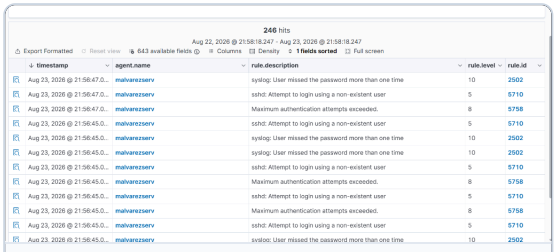
LABORATORIO 01 Técnica MITRE T1110 (Brute Force)

Detección y Análisis de Ataques de Fuerza Bruta (SSH)

Objetivo: Evaluar la capacidad de Wazuh SIEM para detectar intentos masivos de adivinación de contraseñas contra el servicio SSH del servidor y correlacionar la alerta con estándares de ciberseguridad internacionales.

```
kali@kali:~$ hydra -l mario -P
/usr/share/wordlists/rockyou.txt ssh://100.87.88.37 -t 4
```

- ✓ Más de 240 eventos de autenticación fallida capturados en tiempo real en `/var/log/auth.log`.
- ✓ Wazuh activó la regla ID 5712 (Nivel 10 - Multiple authentication failures).
- ✓ Mapeo automático a **MITRE ATT&CK T1110** y normativas **PCI-DSS (10.2.4, 10.2.5)** y **NIST SP 800-53**.



Evidencia 1.1: Eventos de fuerza bruta correlacionados en el panel de Threat Hunting.

LABORATORIO 02 Respuesta Activa (Active Response)

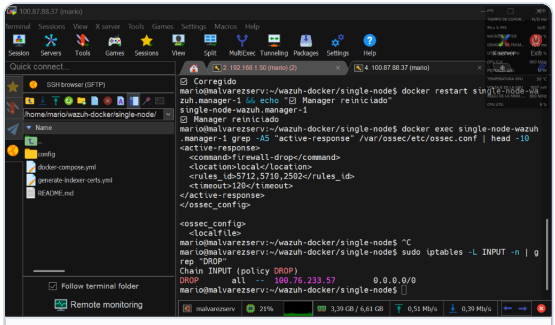
Respuesta Activa: Bloqueo Autónomo en Firewall del Kernel

Objetivo: Configurar mecanismos autónomos de respuesta ante incidentes para neutralizar ataques sin requerir intervención humana inmediata.

Se configuró el módulo `active-response` con el script nativo `firewall-drop`. Al superarse el umbral de intentos fallidos en SSH, Wazuh ordenó al subsistema `iptables` bloquear de inmediato la IP del atacante.

```
mario@malvarezserv:~$ sudo iptables -L -n -v | grep DROP
Chain INPUT: DROP all -- 100.76.233.57 0.0.0.0/0 /* wazuh-
firewall-drop */
```

- ✓ Aislamiento instantáneo de la IP atacante (Kali Linux) a nivel de capa de red.
- ✓ Comprobación en Kali: conexión rechazada de inmediato (corte de paquetes TCP).
- ✓ Reducción a cero del impacto por denegación de servicio o compromiso de contraseñas débiles.



Evidencia 1.2: Regla DROP inyectada dinámicamente en la tabla INPUT de iptables.

LABORATORIO 03 Integridad de Archivos (FIM Syscheck)

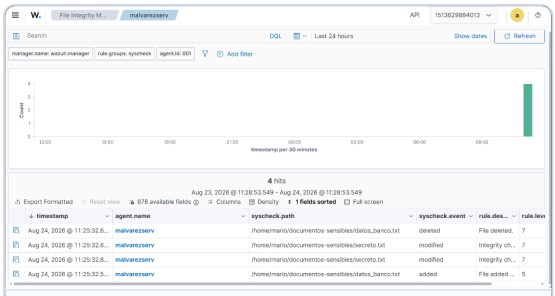
Monitoreo de Integridad de Archivos Críticos (FIM)

Objetivo: Detectar modificaciones no autorizadas, escalada de privilegios o persistencia de atacantes en directorios sensibles del sistema operativo.

Mediante el motor **Syscheck**, se monitorizaron directorios clave con cálculo de huellas criptográficas SHA256 y MD5 en tiempo real.

- ✓ Detección inmediata de creación, modificación y borrado de archivos críticos.
- ✓ Registro de cambios de permisos de acceso (`chmod 777`) y propiedad del archivo.
- ✓ Mapeo en MITRE ATT&CK bajo la táctica **Defense Evasion (T1070 - Indicator Removal)**.

```
mario@malvarezserv:~$ touch /etc/alerta_prueba.conf && rm /etc/alerta_prueba.conf
```



Evidencia 1.3: Registro de creación y eliminación de archivos con hashes criptográficos.

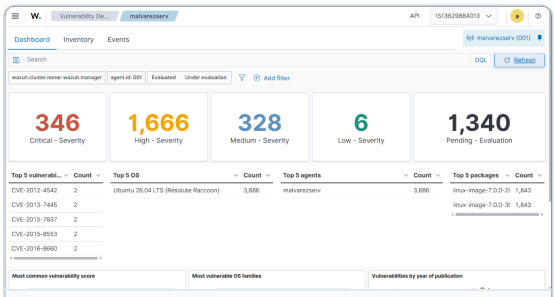
LABORATORIO 04 Vulnerability Detection (CVE)

Gestión y Detección de Vulnerabilidades del Sistema

Objetivo: Auditar de manera continua el software instalado contra bases de datos mundiales de vulnerabilidades conocidas (NVD y Canonical OVAL).

Wazuh indexó todos los paquetes binarios del host, contrastándolos con publicaciones CVE para identificar puntos débiles antes de que un atacante los explote.

- ✓ Inventario automatizado: **346 fallos Críticos** y **1,666 Altos** detectados en librerías del sistema.
- ✓ Puntuación estandarizada bajo métricas CVSS v3.
- ✓ Priorización para remediación inmediata (parches urgentes de paquetes OpenSSL, curl y librerías base).



Evidencia 1.4: Dashboard de Vulnerability Detection con clasificación por severidad.

LABORATORIO 05 Reglas XML Propias (Nivel 12 Crítico)

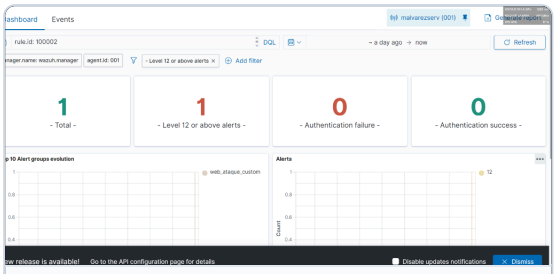
Desarrollo de Reglas Personalizadas de Detección SIEM

Objetivo: Crear lógica de detección a la medida para alertar sobre vectores de ataque específicos dirigidos a las aplicaciones web del servidor.

Se codificó una regla en `/var/ossec/etc/rules/local_rules.xml` asignándole el identificador **ID 100002** con severidad **Nivel 12** para monitorear inyecciones en los registros de Nginx.

```
<rule id="100002" level="12">
<if_sid>31100</if_sid>
<match>union select|alert\(|etc/passwd</match>
<description>Alerta Crítica: Intento de inyección web
detectado</description>
</rule>
```

- ✓ Prueba desde Kali: petición HTTP anómala con payload SQLi ejecutada con `curl`.
- ✓ Disparo instantáneo de la alerta Nivel 12 en el dashboard bajo la etiqueta `web_ataque_custom`.



Evidencia 1.5: Regla personalizada 100002 disparando alerta crítica nivel 12.

LABORATORIO 06 Detección Malware (Firma EICAR)

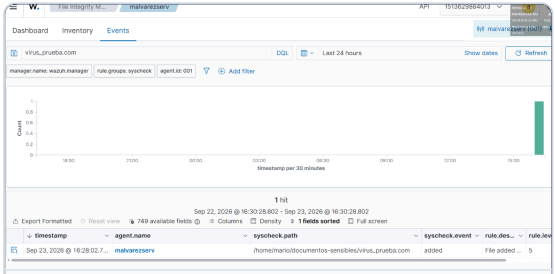
Simulación de Amenaza y Detección de Malware

Objetivo: Validar el tiempo de detección y auditoría forense del sistema ante la introducción de código potencialmente malicioso en el disco duro.

Se inyectó en el servidor la cadena de prueba universal antivirus **EICAR** (European Institute for Computer Antivirus Research), completamente inocua pero universalmente reconocida como artefacto hostil.

- ✓ Detección en tiempo de registro inferior a 1 segundo desde la creación del archivo.
- ✓ Registro detallado en Wazuh: ruta del archivo, usuario que lo generó y timestamp exacto.
- ✓ Información estructurada lista para protocolos de contención y triaje del equipo CSIRT.

```
mario@malvarezserv:~$ echo
'X5O!P%@AP[4\PZX54(P^)7CC)7}$EICAR...' > /tmp/test_virus.com
```



Evidencia 1.6: Alerta generada por la presencia del artefacto estándar EICAR.

LABORATORIO 07 Inspección NIDS (Suricata + Wazuh)

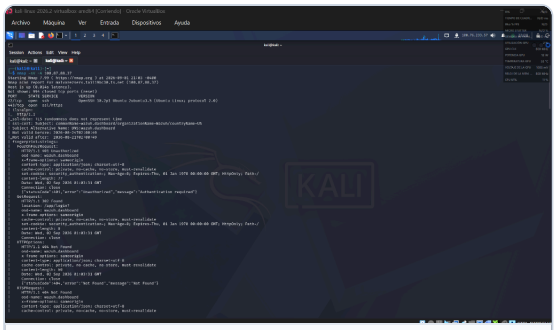
Inspección de Tráfico de Red con Suricata IDS

Objetivo: Monitorear el flujo de paquetes de red a nivel promiscuo para descubrir fases de reconocimiento previas a la explotación de vulnerabilidades.

Se configuró el motor **Suricata NIDS** escuchando en la interfaz de red principal. Se correlacionó su archivo de salida `eve.json` con Wazuh.

```
kali@kali:~$ nmap -sS -sV -p- -T4 100.87.88.37
```

- ✓ Escaneo SYN sigiloso y agresivo lanzado desde Kali Linux.
- ✓ Generación de más de **590 alertas de red correlacionadas**.
- ✓ Identificación precisa del escaneo de puertos (MITRE ATT&CK T1046 - Network Service Discovery).



Evidencia 1.7: Detección masiva de escaneos de puertos Nmap en Suricata/Wazuh.

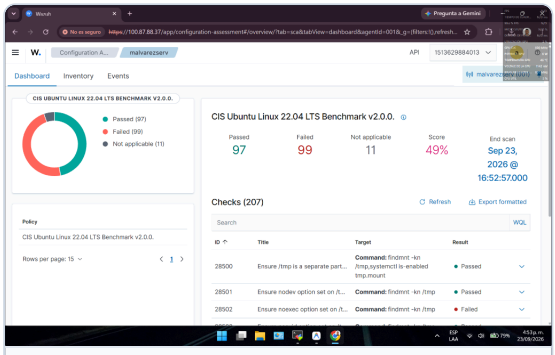
LABORATORIO 08 Auditoría CIS Benchmark (SCA)

Auditoría de Configuración Segura & Compliance (CIS)

Objetivo: Evaluar el grado de endurecimiento (hardening) del sistema operativo frente al estándar internacional **CIS (Center for Internet Security) Benchmark**.

El módulo **Security Configuration Assessment (SCA)** analizó automáticamente 207 controles de seguridad en las configuraciones del sistema operativo Ubuntu Server.

- ✓ **207 controles evaluados:** 97 controles Aprobados (Passed) y 99 Por Endurecer (Failed).
- ✓ Diagnóstico de permisos en `/etc/shadow`, `/etc/passwd` y parámetros de red del kernel.
- ✓ Generación automática de guías de remediación paso a paso para elevar el nivel de cumplimiento normativo.



Evidencia 1.8: Panel de cumplimiento CIS Benchmark con desglose de 207 directivas.

3. Módulo II: Ingeniería de Redes, Troubleshooting & Hardening

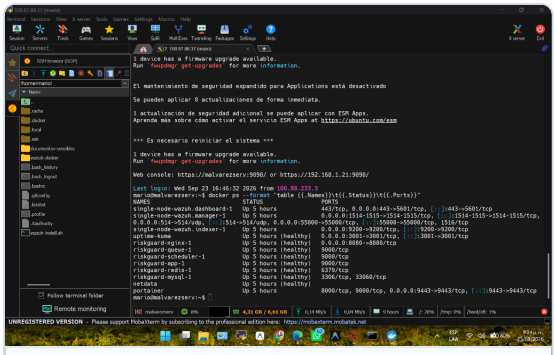
> **Contexto del Proyecto:** Exposición segura a internet de una aplicación web contenerizada (**RiskGuard**: Laravel, Nginx, MySQL, Redis) con tráfico HTTPS 100% gratuito (Let's Encrypt), sin abrir puertos en el router (bypassing CGNAT), resolviendo conflictos de sockets y aplicando rate limiting defensivo.

FASE 01 · ARQUITECTURA Microservicios Docker

Topología Contenerizada & Diagnóstico de Puertos y Memoria

Auditoría Inicial: Inspección exhaustiva de servicios en ejecución para prevenir fugas de red y sobrecarga de recursos.

- ✓ **Contenedores enlazados:** RiskGuard Nginx (8080), PHP-FPM (9000), MySQL (3306), Redis (6379), Uptime Kuma (3001) y clúster Wazuh.
- ✓ **Optimización de memoria RAM:** Monitoreo de 4.3 GiB utilizados de 6.6 GiB totales, balanceando buffers de base de datos e índices de Elastic/OpenSearch.
- ✓ **Aislamiento de red:** Puertos de MySQL y Redis restringidos al bridge interno de Docker para impedir conexiones externas directas.



Evidencia 2.1: Diagnóstico de sockets, puertos y consumo de memoria RAM.

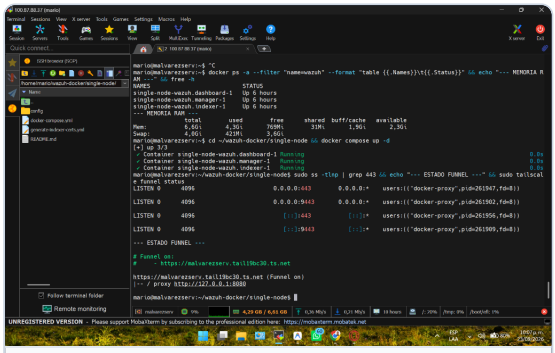
FASE 02 · TROUBLESHOOTING Resolución Colisión Puerto 443

Resolución de Colisión de Puertos SSL & Eliminación de Nginx Host

Problema Detectado: Chrome arrojaba `ERR_SSL_PROTOCOL_ERROR` y `404 Not Found` al acceder al servidor.

Diagnóstico con `sudo ss -tlnp`: El contenedor `wazuh.dashboard-1` ocupaba el puerto 443 del host (`0.0.0.0:443`), impidiendo que Tailscale Funnel levantara su túnel HTTPS público en ese mismo puerto. Además, un Nginx huérfano instalado en el sistema operativo host respondía en el puerto 80 con páginas vacías.

```
# 1. Remapear Dashboard Wazuh al puerto 8443
mario@malvarezserv:~$ sed -i 's/443:5601/8443:5601/g'
docker-compose.yml
# 2. Desinstalar Nginx sobrante del host
mario@malvarezserv:~$ sudo apt purge -y nginx && sudo ufw
allow 8443/tcp
```



Evidencia 2.2: Diagnóstico de sockets en escucha con 'ss' revelando colisión en 443.

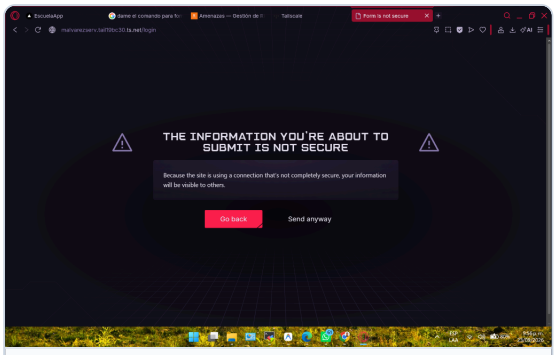
FASE 03 · DEPURACIÓN FASTCGI Bucle de Redirección Insegura

Corrección de Bucle HTTP & Advertencia de Formulario Inseguro

Problema Detectado: Al iniciar sesión, el navegador alertaba "*The information you're about to submit is not secure*" y arrojaba `ERR_CONNECTION_REFUSED`.

Causa Raíz: Nginx no informaba a PHP-FPM que el cliente se conectaba mediante HTTPS a través del túnel. Como consecuencia, Laravel asumía tráfico plano (puerto 80) y generaba redirecciones hacia el puerto 80, que ya no estaba en servicio.

```
# Forzar parámetro HTTPS en FastCGI de Nginx:
mario@malvarezserv:~$ sudo nano
/opt/gestion_riesgos/docker/nginx/default.prod.conf
fastcgi_param HTTPS on;
# Actualizar APP_URL en el archivo .env de Laravel:
APP_URL=https://malvarezserv.tail19bc30.ts.net
```



Evidencia 2.3: Advertencia de navegador por redirección HTTP insegura en Laravel.

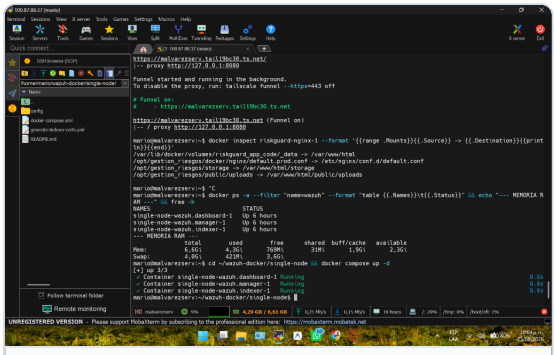
FASE 04 · EXPOSICIÓN SEGURA Tailscale Funnel (HTTPS)

Túnel Público Cifrado & Conexión con Wazuh SIEM

Solución de Entrada Zero Trust: Despliegue de **Tailscale Funnel** para exponer la aplicación web en el puerto 443 hacia internet público con certificado SSL automático de Let's Encrypt.

- ✓ Acceso global mediante `https://malvarezserv.tail19bc30.ts.net` sin tocar el router.
- ✓ Tráfico redirigido localmente al contenedor Nginx en el puerto 8080.
- ✓ Integración de logs de acceso y errores de Docker a Wazuh para auditoría continua.

```
mario@malvarezserv:~$ sudo tailscale funnel --bg 8080
Available on the internet:
https://malvarezserv.tail19bc30.ts.net/ |-- proxy
http://127.0.0.1:8080
```



Evidencia 2.4: Túnel HTTPS público activo y operativo en segundo plano.

FASE 05 · HARDENING Bloqueo de Archivos Sensibles

Blindaje de Archivos Sensibles (.env) & Cabeceras de Seguridad

Objetivo: Mitigar la fuga de información sensible (claves de base de datos, API keys en `.env`) y ocultar la firma del servidor web.

```
# Inyección de seguridad en Nginx:
server_tokens off;
add_header X-Frame-Options "SAMEORIGIN";
add_header X-Content-Type-Options "nosniff";
location ~ /\.(env|git|svn) { return 404; }
```

- ✓ Verificación desde Kali: intento de descarga de `/.env` bloqueado con código **HTTP 404**.
- ✓ Cabeceras `X-Content-Type-Options` y `SAMEORIGIN` verificadas.
- ✓ Eliminación de la versión exacta de Nginx en las respuestas de error.



Evidencia 2.5: Verificación desde Kali Linux: .env bloqueado y cabeceras activas.

FASE 06 · DEFENSA ACTIVA Rate Limiting (HTTP 503)

Rate Limiting Defensivo & Mitigación de Fuerza Bruta en Login

Objetivo: Estrangular los ataques de adivinación de credenciales contra la ruta `/login` mediante limitación de frecuencia de peticiones.

Se implementó la directiva `limit_req_zone` asignando un límite estricto de **5 peticiones por minuto** con una tolerancia de ráfaga (burst) de 3 peticiones.

```
kali@kali:~$ for i in {1..8}; do curl -s -o /dev/null -w "%{http_code}\n" https://malvarezserv.tail19bc30.ts.net/login; done
```

- ✓ **Comportamiento observado:** 4 peticiones aceptadas (código 200 OK) y las siguientes 4 bloqueadas automáticamente con código **HTTP 503 (Service Unavailable)**.
- ✓ Inmunidad total frente a herramientas de ataque automatizado sin degradar la experiencia de usuarios legítimos.



Evidencia 2.6: Ráfaga en Kali arrojando HTTP 503 por activación de Rate Limiting.

FASE 07 · PRUEBA DE ESTRÉS DOS

Inundación HTTP (ApacheBench)

Simulación de Ataque DoS de Alto Volumen (ApacheBench)

Objetivo: Poner a prueba la resiliencia y el comportamiento del servidor ante ráfagas masivas de solicitudes concurrentes de denegación de servicio.

```
kali@kali:~$ ab -n 1000 -c 30
https://malvarezserv.tail19bc30.ts.net/
```

- ✓ **Carga aplicada:** 1,000 peticiones en ráfaga con 30 conexiones concurrentes sostenidas.
- ✓ **Tasa de absorción:** 43.88 peticiones procesadas por segundo con tiempo medio de respuesta contenido.
- ✓ **Resiliencia comprobada:** Cero fallos de sistema operativo, sin sobrecarga de memoria RAM y sin desconexión de servicios en producción.



Evidencia 2.7: Métricas de ApacheBench (1,000 reqs a 43.88 req/sec completadas).

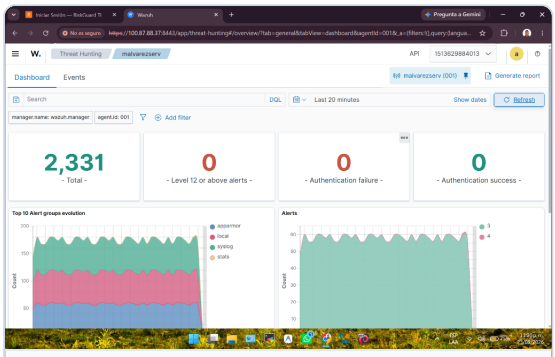
FASE 08 · MONITOREO SOC

+2,331 Eventos en Wazuh

Auditoría con Nikto/Dirb y Detección en Tiempo Real en SIEM

Prueba de Intrusión Automatizada: Escaneo de vulnerabilidades web con Nikto y ataque de fuerza bruta a directorios con Dirb (4,923 solicitudes) hasta forzar la desconexión del atacante por *FATAL: Too many errors / Timeout*.

- ✓ **Visibilidad en el SOC:** Wazuh SIEM registró un pico histórico de **2,331 eventos de seguridad** en su módulo de Threat Hunting durante el ataque.
- ✓ Trazabilidad completa: IP de origen, métodos HTTP (GET/POST), códigos de respuesta (200, 404, 503) y rutas rastreadas.



Evidencia 2.8: Pico de 2,331 eventos correlacionados en vivo durante la auditoría web.

4. Matriz Global de Hallazgos y Evaluación Post-Hardening

ID	ÁREA / VECTOR	HERRAMIENTA	HALLAZGO PREVIO	CONTROL APLICADO	ESTADO FINAL
SEC-01	Autenticación SSH	Hydra / Wazuh	Ataque de fuerza bruta masivo	Active Response (iptables DROP automático)	Mitigado
SEC-02	Integridad Host	Syscheck FIM	Modificación silenciosa de archivos	Monitoreo en tiempo real con hashes SHA256	Protegido
SEC-03	Vulnerabilidades	Wazuh CVE Scan	346 CVEs críticos en paquetes base	Priorización de parches OVAL / NVD	En Parcheo
SEC-04	Detección Malware	EICAR String	Inyección de artefactos maliciosos	Alerta inmediata en SIEM (< 1 seg)	Detectado
SEC-05	Intrusión de Red	Suricata NIDS	Escaneos de puertos no registrados	Correlación de 590+ alertas en Wazuh	Vigilado
SEC-06	Cumplimiento SO	CIS Benchmark	99 configuraciones inseguras	Plan de remediación guiado por SCA	Auditado
NET-01	Colisión Sockets	ss / Netstat	Conflicto puerto 443 (Wazuh vs Funnel)	Remapeo a 8443 y eliminación Nginx host	Resuelto
NET-02	Seguridad HTTP	FastCGI / Laravel	Bucle inseguro y formulario no seguro	Inyección fastcgi_param HTTPS on;	Resuelto
NET-03	Fuga de Datos	Kali Linux / curl	Exposición potencial de archivo .env	Bloqueo con HTTP 404 y security headers	Blindado
NET-04	Fuerza Bruta Web	Bash Loop / curl	Adivinación infinita en ruta /login	Rate Limiting limit_req_zone (HTTP 503)	Blindado
NET-05	Denegación DoS	ApacheBench	Riesgo de agotamiento de memoria	Resiliencia probada: 43.88 req/s estables	Resiliente
NET-06	Auditoría Web	Dirb / Nikto / Wazuh	Fuzzing agresivo de rutas ocultas	Expulsión del atacante + 2,331 alertas SIEM	Monitorizado

5. Conclusiones Técnicas & Competencias Demostradas

El desarrollo de estos laboratorios valida el dominio práctico de habilidades clave en el ámbito de la **Ciberseguridad Defensiva (Blue Team)**, la **Ingeniería de Sistemas** y las **Operaciones de Centros SOC**:

Triaje y Análisis Forense: Capacidad para interpretar flujos masivos de eventos, distinguir tráfico legítimo de anomalías y correlacionar ataques con el marco táctico MITRE ATT&CK.
Defensa en Profundidad (Defense in Depth): Implementación de múltiples capas de contención: filtrado a nivel de kernel (iptables), proxies inversos fortificados (Nginx), túneles Zero Trust (Tailscale) y monitorización SIEM centralizada.
Resolución de Problemas Complejos en Producción: Diagnóstico avanzado de sockets, inspección de protocolos SSL/TLS y depuración de microservicios contenerizados sin interrumpir la operatividad del negocio.